Making KePIN Better

An easy guide to what we did, what we found, and **why**

Remaining-useful-life prediction for jet engines (NASA C-MAPSS) · 18–21 October 2026

The whole story in one paragraph

We started out trying to improve KePIN by making its Koopman part "dynamic". By explaining what the model was really doing, we found that KePIN's Koopman part was not driving its accuracy, and that the real problem on the hardest datasets was **flying-condition noise** drowning out the **wear signal**.

Fixing that inside the architecture (a **condition-aware KePIN**) beat the original KePIN and the published paper by **1.1 RMSE on FD002** and **1.8 RMSE on FD004**. Along the way we built a **wear model** that matches KePIN's accuracy and explains every single prediction.

How to read this guide

- Each section follows the same pattern: **what we did**, the **proof** (a figure, real code or a table), a yellow **Why?** box, and what it means.
- Blue boxes give the short version. You can read just the blue and yellow boxes for a quick overview.
- All numbers are RMSE (how many flights off the predictions are, on average). **Lower is better.**

Contents

1. KePIN architecture and RUL prediction
2. Why Koopman wasn't helping: the feature replacement experiment
3. Koopman ablation study across FD001–FD004
4. Signal A explainability: operating conditions vs engine wear
5. Why dynamic Koopman failed
6. The time-to-failure (TTF) model and its failure
7. The wear model: wear accumulator and failure threshold
8. Wear model explainability and validation
9. Condition-aware KePIN: the final architecture
10. Condition-aware normalisation and EMA smoothing
11. Koopman with control
12. Final results, ablation and limitations
13. Glossary

1. KePIN Architecture and RUL Prediction

In one sentence

KePIN reads the last 30 flights of an engine's sensors and predicts how many flights it has left; it has a "Koopman" part that is supposed to model how the engine changes over time.

The problem: Remaining Useful Life (RUL)

Every jet engine slowly wears out. If we can predict how many flights an engine has left before it fails, airlines can service it just in time: not too early (wasteful), not too late (dangerous). That number is the **Remaining Useful Life (RUL)**.

The data: NASA C-MAPSS

C-MAPSS is a simulated fleet of engines. Each engine has 21 sensors (temperatures, pressures, speeds) and 3 operating settings, recorded once per flight until the engine fails. There are four sub-datasets:

Dataset	Flying conditions	Failure types	Difficulty	Paper's KePIN RMSE
FD001	1	1	easiest	12.92
FD002	6	1	hard	15.08
FD003	1	2	medium	11.42
FD004	6	2	hardest	17.04

A **flying condition** is a combination of altitude, speed and throttle. On FD002 and FD004 an engine switches between 6 of them, almost every flight. Remember this: it turns out to be the key to the whole story.

How KePIN makes a prediction

Original KePIN: how one prediction is made

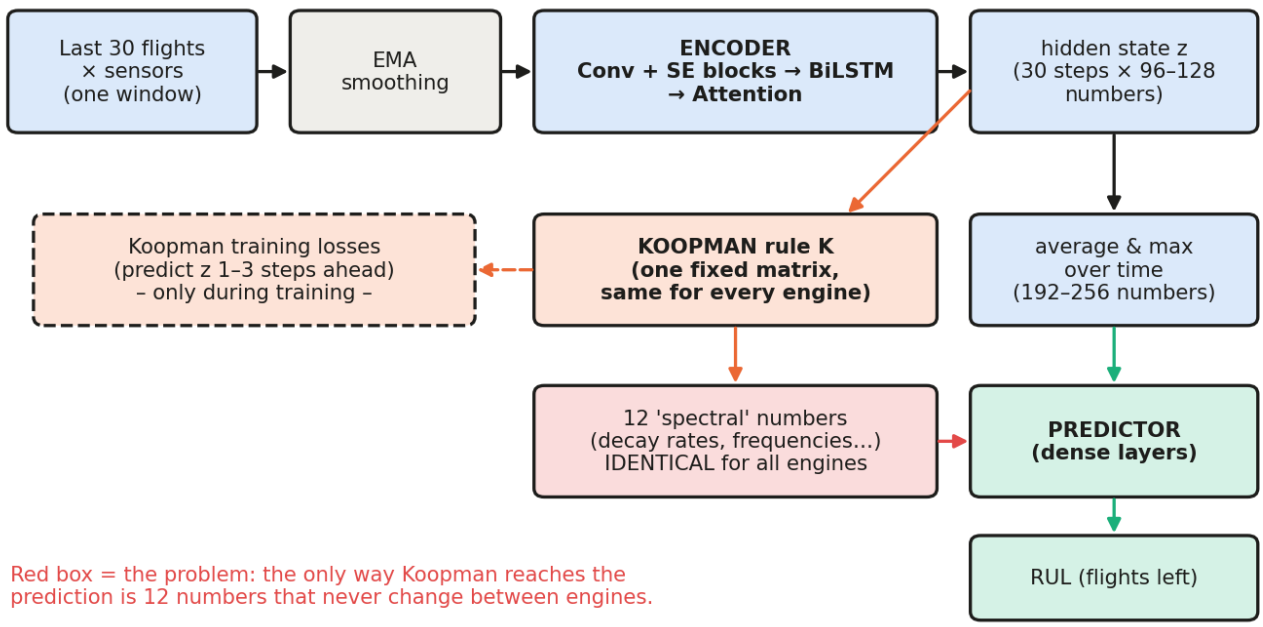


Figure 1. The original KePIN. The encoder turns 30 flights of sensor data into a hidden state z. The predictor reads a summary of z plus 12 numbers from the Koopman rule.

What is the Koopman part? A Koopman operator is a learned "rule of motion": a matrix K that moves the hidden state from one flight to the next (next state = K × current state). It splits the engine's behaviour into simple patterns ("modes") that each fade at their own speed. The idea is elegant: complex, non-linear wear becomes simple, linear motion in the hidden space.

Two things you need to know about how results are measured

- **Average vs ensemble.** We train every model 3 times (3 "seeds"). The **average** is the mean error of the 3 models. The **ensemble** first averages the 3 models' predictions, then scores once (usually a bit better). **The paper reports ensembles**, so that is our headline number.
- **Fair test vs the paper's method.** Training produces many versions of the model; one must be chosen. The original code chooses it using the **test set itself**, which is like checking practice attempts against the real exam answers. We call this the **paper's method**, and we use it so our numbers compare directly with the paper. Choosing on held-out *training* engines instead (the **fair test**) gives errors about 0.4–0.6 higher.

```
history, swa_weights = trainer.fit_enhanced(
    X_train_aug, Y_train_aug, X_test_tuple, Y_test,      # <- the TEST set is passed as "validation"
    ...
if val_rmse < best_val_rmse:                              # <- so "best" means best on the test set
    best_val_rmse = val_rmse
    best_weights = self.model.get_weights()
```

Source: `RCKePIN_Signal_A/train_fd001_conditioned.py:169-170 and kepin_cmapss_optimized.py:486-489`

Why?

We kept the paper's method so our results are an apples-to-apples comparison with the published numbers. Our reproduction of KePIN matches the paper on all four datasets (12.92 / 15.00 / 11.56 / 17.11 vs 12.92 / 15.08 / 11.42 / 17.04), so the baseline we compare against is faithful.

2. Why Koopman Wasn't Helping: the Feature Replacement Experiment

In one sentence
The 12 numbers Koopman gives KePIN's predictor are exactly the same for every engine, so they cannot help tell a healthy engine from a failing one.

What we found in the code

In the original KePIN, the Koopman rule K is **one fixed matrix**, the same for every engine. Its 12 "spectral features" (decay rates, frequencies and so on) are computed once from K, then simply **copied** to every engine in the batch:

```
spec_feats = extract_spectral_features(
    koopman_out["eigenvalues"], top_k=self.spectral_k,    # computed from K only (not from the engine)
)
batch_size = tf.shape(inputs)[0]
spec_feats_batch = tf.tile(                               # the SAME 12 numbers copied to every engine
    tf.expand_dims(tf.cast(spec_feats, tf.float32), 0), [batch_size, 1],
)
head_input = tf.concat([pooled, spec_feats_batch], axis=-1)
```

Source: `original_kepin/code/kepin_model.py:303-311`

The feature replacement experiment

In our *dynamic* Koopman versions (section 5), the 12 numbers *do* differ per engine. So we tested whether the predictor uses them: we took the trained models, **replaced the 12 numbers with a constant**, and measured how much the predictions changed.

Model (FD002)	Error with real features	Error with a constant	Largest change in any prediction
Dynamic Koopman B (health), run 0	15.538	15.538	0.000 flights
Dynamic Koopman A (condition), run 0	16.207	16.208	0.09 flights

Why?
Analogy: a doctor predicts each patient's health from their test results plus a "Koopman note". If the note is the same printed sheet for every patient, it cannot help tell patients apart.
And even when the note *was* different per patient, the predictor had learned to ignore it, because it already received the full hidden state (192–256 numbers) directly.

What it means: in KePIN, Koopman never informs an individual prediction. It can only influence accuracy indirectly, through its training losses shaping the encoder. Section 3 tests whether even that helps.

3. Koopman Ablation Study Across FD001–FD004

In one sentence

We removed Koopman completely from KePIN and retrained it on all four datasets: accuracy did not change significantly anywhere.

What we did

An **ablation** means removing one part and retraining, to see whether that part matters. We built KePIN with **no Koopman layer, no spectral features and no Koopman losses**, keeping everything else identical (same encoder, predictor, training recipe and seeds). Command: `train.py --koopman none`.

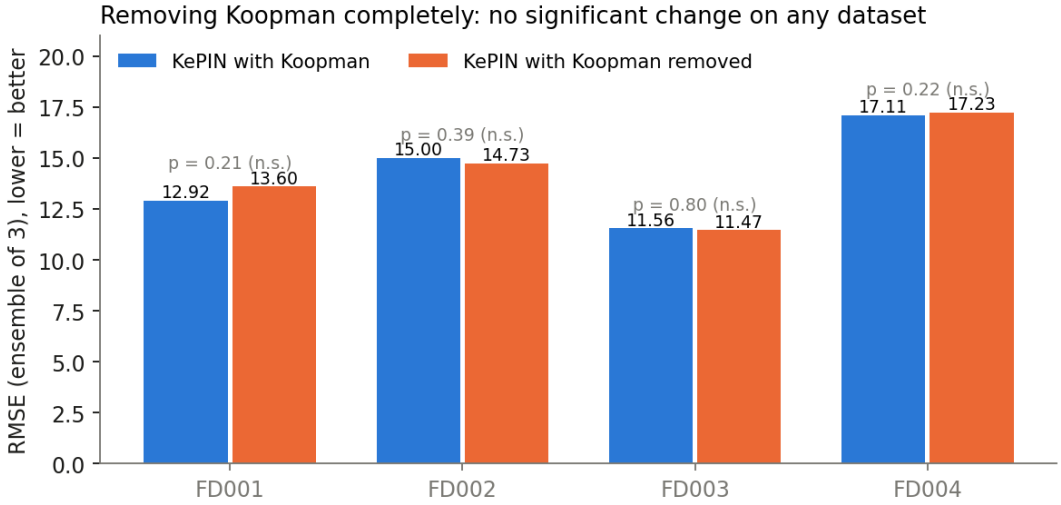


Figure 2. Ensemble of 3 runs, paper's method. p-values compare the two versions seed by seed; below 0.05 would mean a real difference.

Why?

Removing Koopman sometimes helps (FD002, FD003) and sometimes hurts (FD001, FD004), and never significantly. That pattern is what you expect when a part does very little. KePIN's accuracy comes from its encoder (convolution + LSTM + attention), not from Koopman.

Why the original paper never found this

The KePIN repository has an ablation script, but its "without spectral" and "without multi-step" variants set a loss weight to 0 **while also switching on automatic loss weighting**, and the automatic weighting ignores those zeros:

```
{"name": "B: w/o Spectral", "use_auto_weights": True,          # auto-weighting ON ...
 "fixed_weights": {..., "spectral": 0.0, ...}}                 # ... so this 0.0 is never used

if ab_config["use_auto_weights"] and ab_config["use_koopman"]:
    loss_fn = make_kepin_loss(loss_weights_layer=model.loss_weight_layer,
                              use_auto_weights=True)           # fixed_weights not even passed
```

Source: `original_kepin/code/kepin_ablation.py:136-144 and 372-376`

So those variants trained the full model. Its "no Koopman" baseline used a different, smaller network, and its saved results contain no C-MAPSS rows at all. Our ablation is the first controlled test.

4. Signal A Explainability: Operating Conditions vs Engine Wear

In one sentence

The earlier dynamic model ("Signal A") was supposed to react to engine wear, but explaining it showed that it reacted to the flying conditions instead: a shortcut.

What Signal A was

Before this work, the project's dynamic model computed 4 statistics of each 30-flight window (mean, variance, skewness, kurtosis), called **Signal A**, and fed them to a small network that changed the Koopman rule. The claim was that it speeds up the model's "decay" as the engine wears out.

How we tested what it really follows

For every window we computed the **mix of flying conditions** (fraction of its 30 flights in each of the 6 conditions) and the true RUL. Then we asked which of the two predicts Signal A and the network's output better (random forests, always tested on engines they never saw).

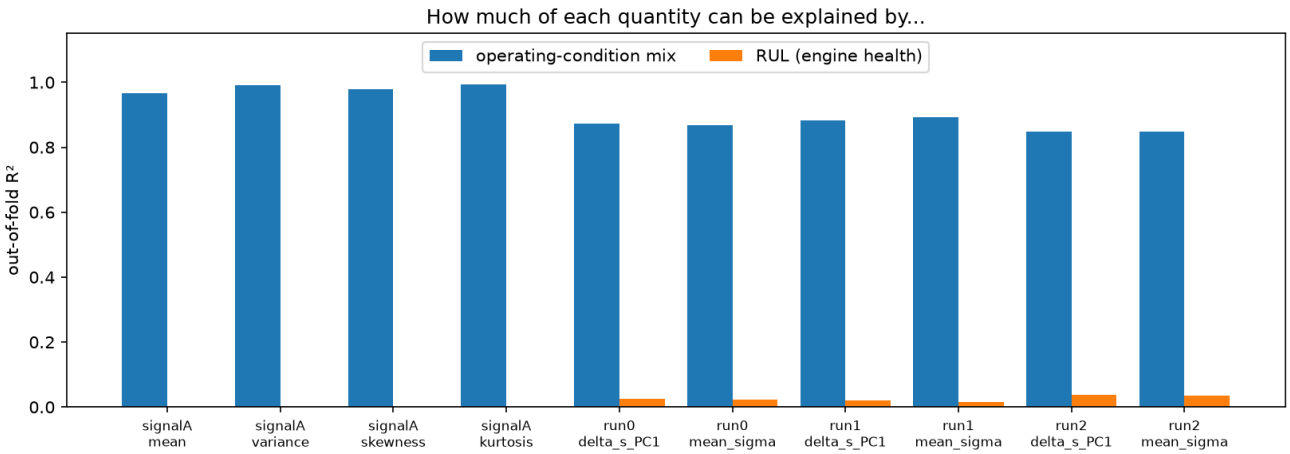


Figure 3. R^2 = how much of each quantity can be explained (1 = all, 0 = nothing). Blue: by the flying-condition mix. Orange: by remaining life (wear). Blue wins everywhere.

Why?

KePIN scales every sensor with **one global average and spread** for all 6 conditions. A window that switches conditions a lot therefore looks "spread out" and "spiky", even for a healthy engine. Signal A's variance and kurtosis mostly measure *condition switching*, so the network learned to follow that.

This is a **shortcut**: the model found an easy pattern that is not the one we wanted. It was only discovered because we explained the model instead of just looking at its RMSE.

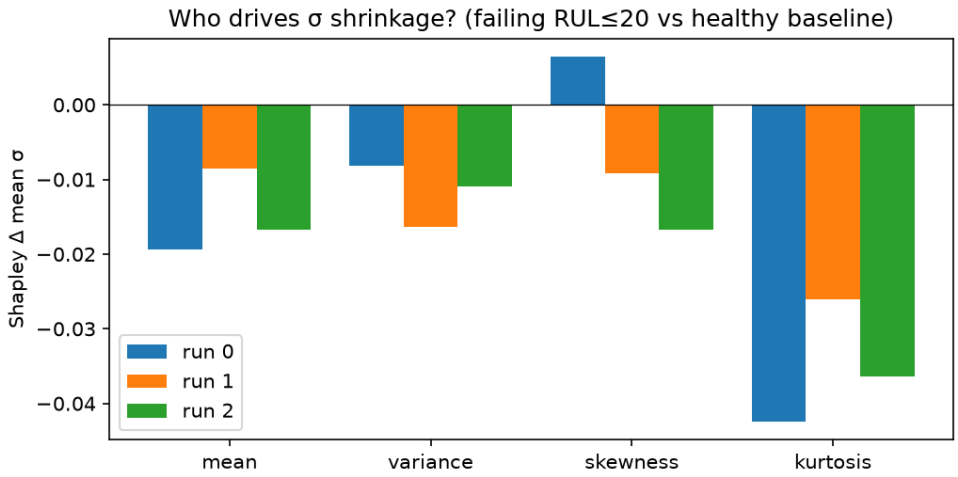


Figure 4. Shapley values (an explainability method that fairly splits the output between inputs): kurtosis, the "spikiness" statistic most sensitive to condition switching, drives the network the most in all 3 runs.

What it means: this finding is the seed of our final improvement (sections 9–10): fix the flying-condition problem *inside* the architecture.

5. Why Dynamic Koopman Failed

In one sentence

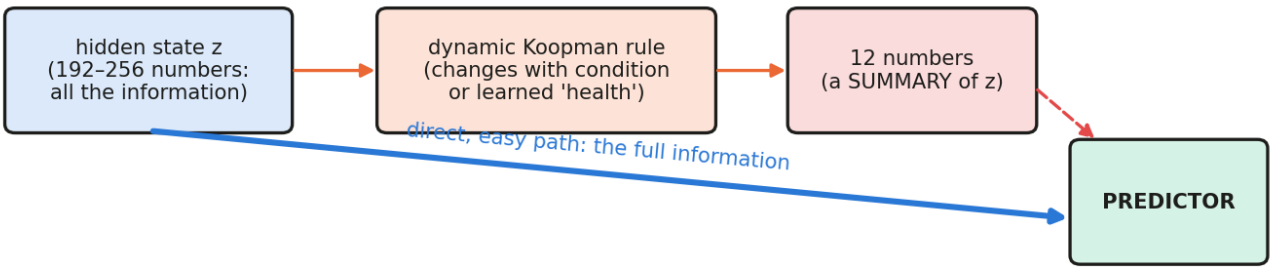
Making the Koopman rule change over time did not help, because it only re-packaged information the predictor already had, and the predictor ignored it.

What we tried (FD002, 3 seeds)

We first built a faster Koopman layer that sets the rule's eigenvalues directly. The original layer had to compute them for every engine on the CPU, which took about 450 s per training epoch. Then we tried three ways of making the rule dynamic:

Version	What the Koopman rule followed	Fair-test RMSE (avg of 3)	vs fixed rule
Fixed rule (baseline)	nothing	15.79	-
A	the flying condition, at every flight	16.16	worse (p = 0.24)
B	a learned "health" value	15.96	no better (p = 0.74)
A + B	both	16.19	worse (p = 0.12)

Why making Koopman 'dynamic' could not help



The dynamic rule only re-packages information the predictor already receives in full. A summary of something you already know adds nothing, so training learns to ignore it (replacing the 12 numbers by a constant changed predictions by < 0.1 flights).

Why?

1. No new information. The dynamic rule was computed from things the predictor already had (the flying condition is visible in the sensors; the "health" came from the same hidden state).

2. It followed the wrong thing. A followed flying conditions, which say nothing about wear. B's "health" was never told what health means: its correlation with true RUL was +0.10, −0.20 and −0.17 in the 3 runs.

3. A weak channel. The 12 numbers involve sorting eigenvalues, which makes learning signals awkward, while the 192–256 encoder numbers are a direct, easy path. Training takes the easy path.

The lesson: "dynamic" is not the same as "useful". To improve predictions, a component must (1) bring information the model does not already have and (2) sit on a path the predictor actually uses.

6. The Time-to-Failure (TTF) Model and Its Failure

In one sentence

We made Koopman the predictor itself ("fast-forward health until it hits the failure line"), but the projected health never reached the line, because a stable Koopman rule can only make things fade.

The idea

If the predictor ignores Koopman, make Koopman **be** the predictor:

- read the engine's current hidden state and turn it into a **health score**;
- use the Koopman rule to **fast-forward** that state flight by flight, up to 125 flights;
- **RUL = number of flights until the projected health crosses the failure line** (0).

What happened

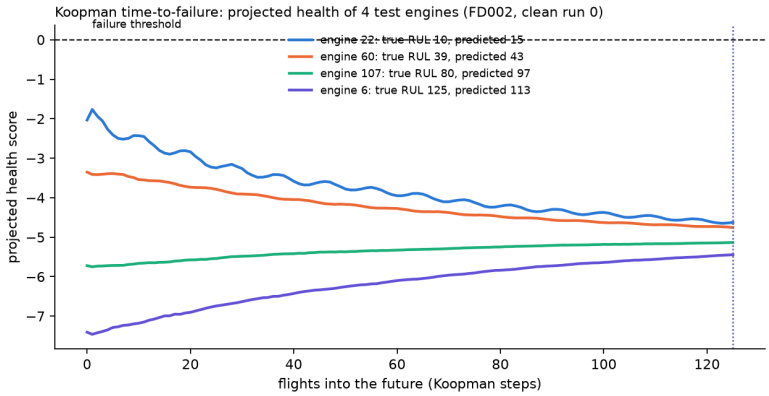


Figure 5. Projected health of 4 test engines (FD002). None of the curves reaches the failure line at 0; they flatten out towards an average level. Across all 253 test engines and 3 runs, 0% crossed.

Accuracy was a little worse than KePIN (fair test 16.44 vs 15.98). The model fell back on reading the *current* health level (correlation −0.91 with RUL) instead of forecasting a crossing.

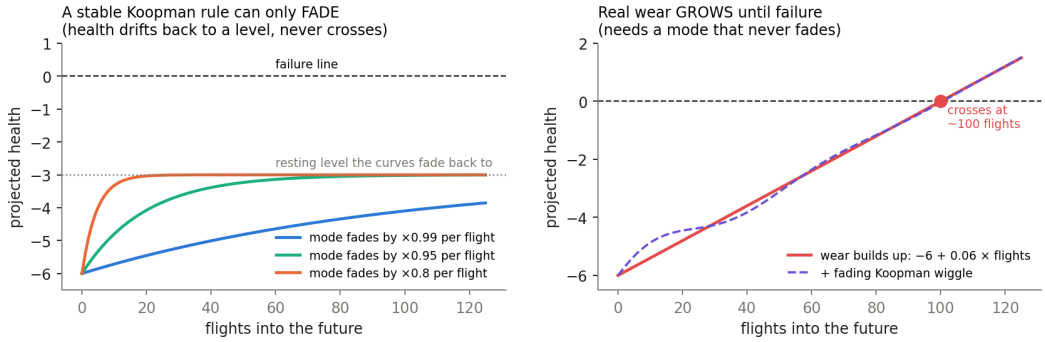


Figure 6. Left: a stable Koopman rule (every mode shrinks by a factor below 1 each flight) can only drift back to a resting level. Right: real wear builds up until it crosses the failure line, which needs a mode that never fades.

Why?

Koopman rules are built to be **stable** (all decay factors below 1), so predictions never explode. But stability means every pattern **fades**. Wear is the opposite: it **accumulates**. A rule that can only fade cannot draw a curve that climbs to failure. This physical mismatch reappears later (section 12, "future loss").

7. The Wear Model: Wear Accumulator and Failure Threshold

In one sentence

Adding one Koopman mode that never fades (the "wear accumulator") lets projected health climb to the failure line, giving a model as accurate as KePIN that explains every prediction.

The wear model: Koopman becomes the predictor

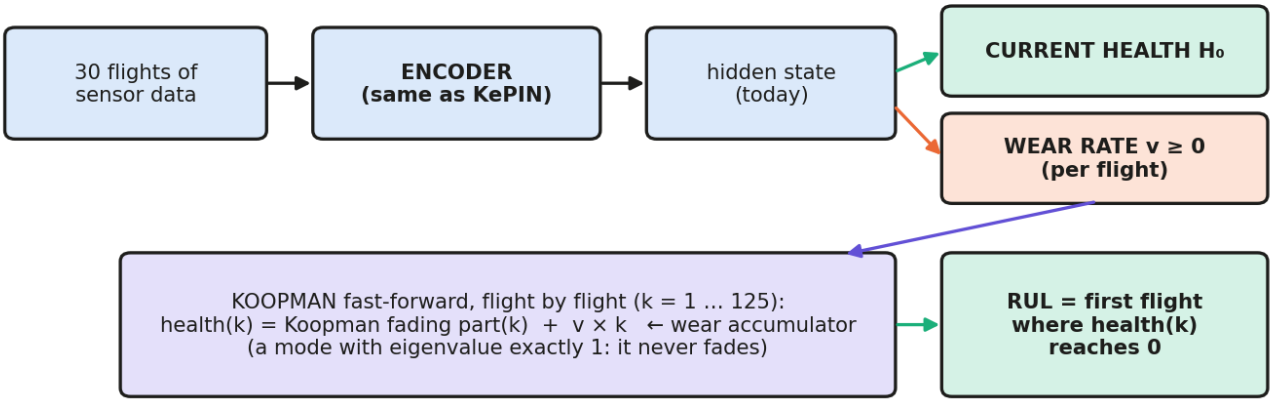


Figure 7. The wear model. The encoder is KePIN's; the black-box predictor is gone. RUL is read directly from the Koopman projection.

The fix in one line of maths

projected health after k flights = Koopman fading part(k) + v × k, where **v ≥ 0** is a learned, per-engine **wear rate**. In Koopman terms, v × k comes from an extra mode with eigenvalue exactly 1 (an "integrator"), which never fades. RUL is the flight where the curve crosses 0, roughly **−current health ÷ wear rate**.

```
v = tf.nn.softplus(tf.tensordot(z_T, self.wear_w, 1) + self.wear_b)[:, None] + 1e-3   # wear rate >= 0
k = tf.range(TTF_HORIZON + 1, dtype=tf.float32)[None, :]                              # flights 0..125
health = health + v * k                                  # wear accumulator mode
logits = -(health[:, 1:] / v) / WEAR_TAU                 # crossing measured in flights
```

Source: `dynamic_koopman/dk_model.py:193-198`

Why?

Measuring the crossing **in flights** (dividing by v) stops the model from cheating as the TTF model did: the "probability still alive" only drops near the real crossing, not gradually from far away.

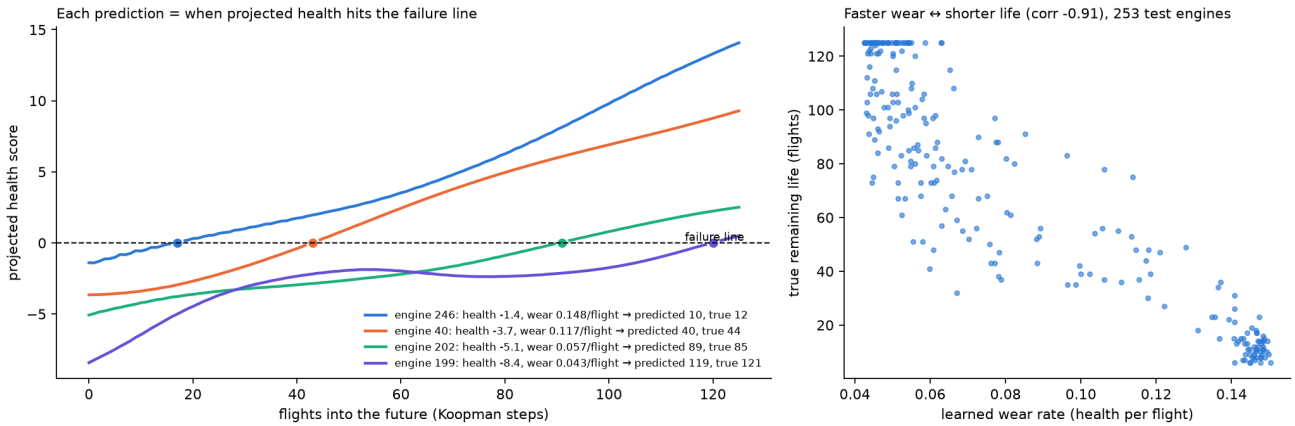


Figure 8. Left: each prediction is literally the point where projected health hits the failure line (dots). Right: the learned wear rate against true remaining life for 253 test engines: faster wear means shorter life (correlation −0.91).

Dataset	Our KePIN	Wear model	Paper's KePIN
FD001	12.92	13.43	12.92
FD002	15.00	14.87	15.08
FD003	11.56	11.43	11.42
FD004	17.11	16.98	17.04

Ensemble of 3, paper's method. No difference is statistically significant: the wear model ties KePIN while explaining every prediction.

8. Wear Model Explainability and Validation

In one sentence

Three independent checks show the wear rate is real: it rises over every engine's life, it ignores flying conditions, and it is driven by the sensors known to show wear.

Check 1: how the wear rate changes over time

We ran the model at every flight of 52 engines it never trained on, from the first flight to failure.

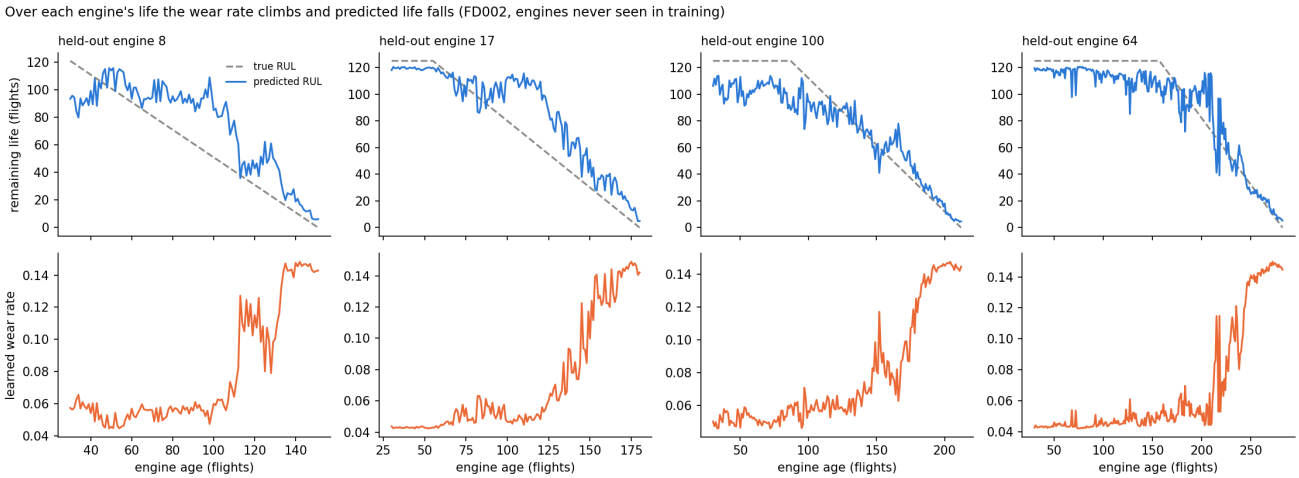


Figure 9. Top: predicted (blue) vs true (dashed) remaining life. Bottom: the learned wear rate over the engine's life.

- The wear rate rose over life in **100%** of the 52 engines (median trend strength +0.92).
- Predicted life fell in **100%** of them (median −0.92).
- The wear rate has **two phases**: flat and low (about 0.05) while healthy, then a sharp climb (to about 0.14) once degradation starts. The model **found the start of degradation by itself**.

Check 2: did it learn the flying-condition shortcut again?

What explains it? (R^2)	From true remaining life	From the flying-condition mix
Signal A network (old, section 4)	0.02–0.04	0.85–0.88 (shortcut)
Wear rate (new)	0.85–0.87	−0.13 (no information)

Check 3: which sensors drive the wear rate?

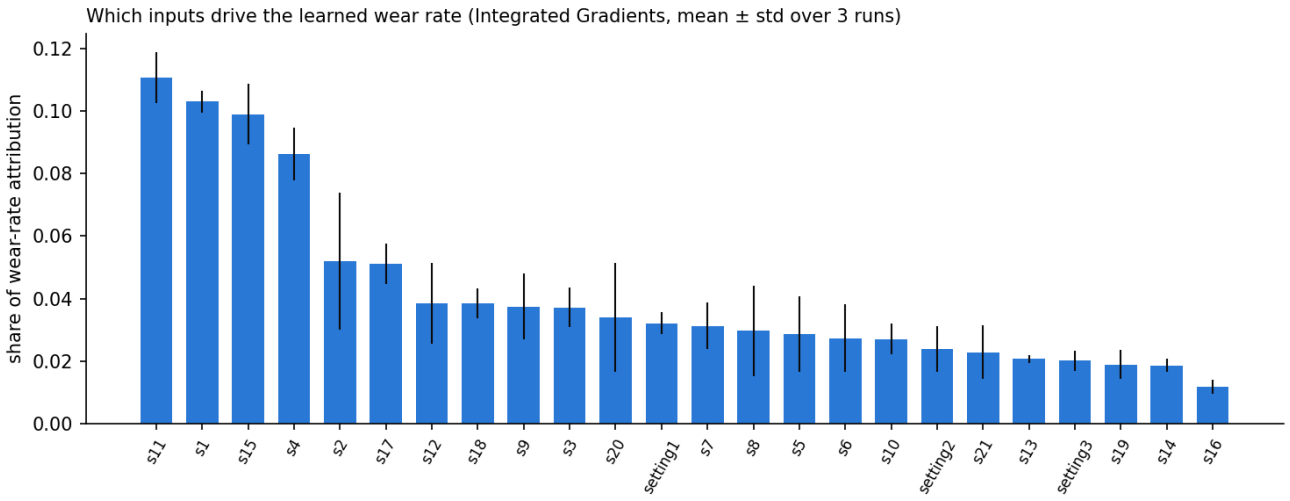


Figure 10. Integrated Gradients: how much each input contributes to the wear rate (mean ± spread over 3 runs).

Why?

The top drivers (**s11** compressor pressure, **s15** bypass ratio, **s4** turbine temperature, s2, s17, s12) are the sensors known to show degradation in C-MAPSS, and the model found them without being told. The 3 flying-condition settings together get only 7.6%.

s1 (fan inlet temperature) ranks high but depends mostly on the flying condition. Since the wear rate shows no condition dependence (check 2), the model most likely uses s1 to judge the other sensors *relative to the current condition*.

Explanations held on every dataset: 85–100% of engines cross the failure line, and the wear rate correlates −0.88 to −0.95 with true remaining life on FD001–FD004.

9. Condition-Aware KePIN: the Final Architecture

In one sentence

We fixed, inside the architecture, the flying-condition problem found in section 4, and it beat KePIN and the paper by 1.1–1.8 RMSE on FD002 and FD004, without hurting FD001 and FD003.

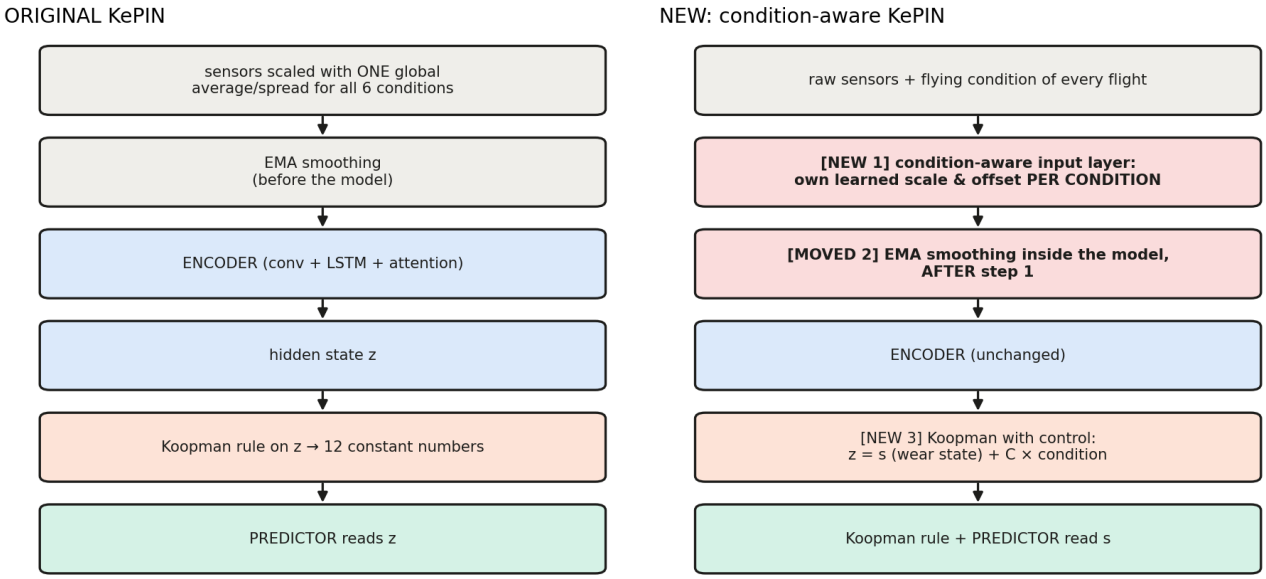


Figure 11. Original KePIN (left) vs condition-aware KePIN (right). Only the red and orange parts changed; the encoder and predictor are KePIN's.

Change	Before (KePIN)	After (new)	Why
1. Condition-aware input layer (new)	one global scaling for all 6 conditions	own learned scale & offset per condition	removes the jumps caused by condition switching (section 10)
2. Smoothing moved (changed)	EMA smoothing before the model	EMA inside the model, after change 1	smoothing first blends different conditions together (section 10)
3. Koopman with control (new)	Koopman rule on z	z = s (wear state) + C × condition	intended to give Koopman a condition-free state (section 11)

Safety check: is the comparison fair?

With the new parts set to neutral (scale 1, offset 0, C = 0), the model gives **exactly the same predictions as original KePIN** (difference 0.000006), and moving the smoothing inside the model changes the inputs by only 0.0000004. So any improvement comes **only** from the new parts.

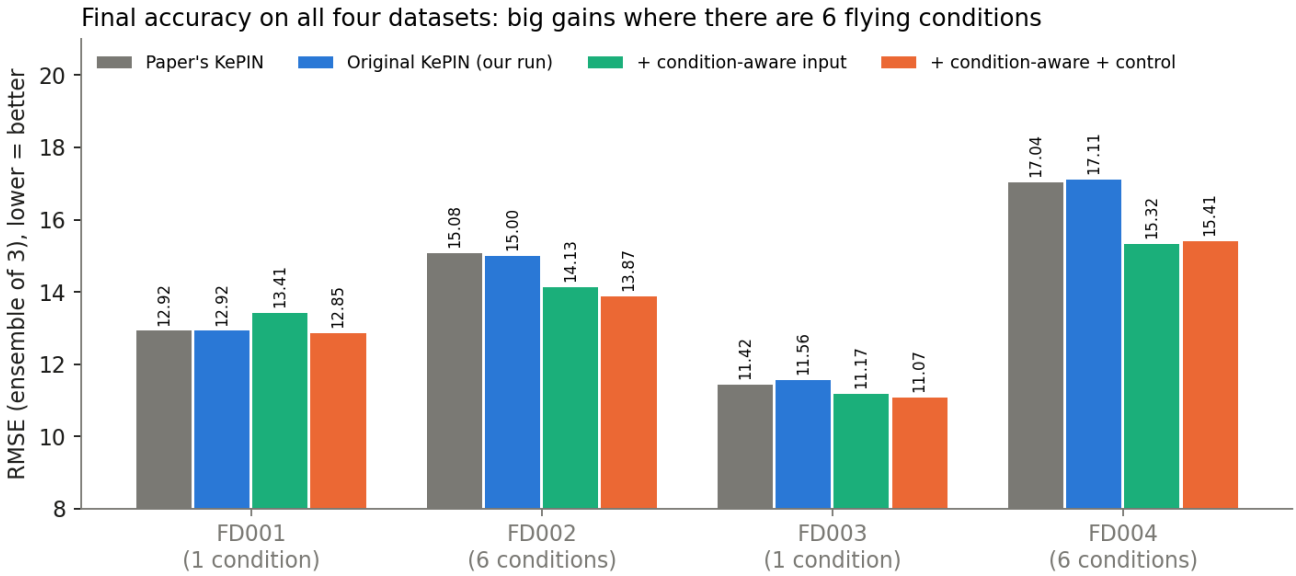


Figure 12. Ensemble of 3 runs, paper's method. On FD002 and FD004 all 12 new runs beat KePIN with the same seed (p = 0.003–0.05), and the NASA score (which punishes late predictions most) improved: FD002 869 → 825, FD004 1265 → 1043. On FD001 and FD003 (one flying condition) the changes are small and not significant, as expected.

10. Condition-Aware Normalisation and EMA Smoothing

In one sentence

Scaling each sensor separately for each flying condition, before smoothing, turns a jumpy signal into a clean wear trend; that is where the accuracy gain comes from.

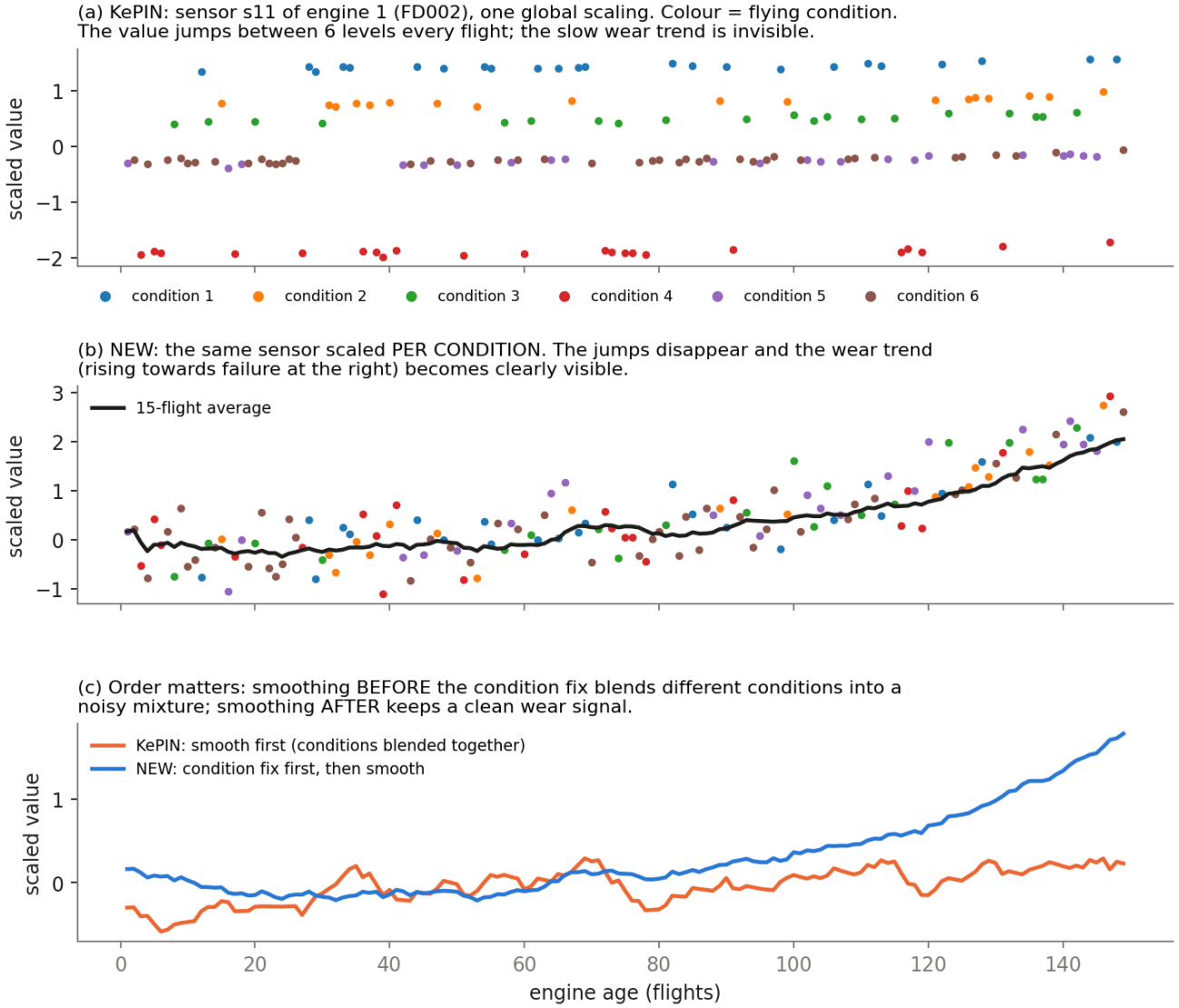


Figure 13. Real data: sensor s11 of engine 1 in FD002, coloured by flying condition.

Why?

(a) The problem. With one global scaling, the same sensor sits at a completely different level in each flying condition. Every flight the value jumps between these levels, and the jumps are far larger than the slow drift caused by wear.

(b) The fix. Scaling per condition asks "how unusual is this reading *for this flying condition*?". The jumps vanish and the wear trend appears clearly, rising towards failure.

(c) Order matters. KePIN smooths each window first. When conditions switch every flight, smoothing averages readings from different conditions into a meaningless mixture. Doing the condition fix first and smoothing afterwards keeps the trend.

How it is built

```
x, u_flat = inputs                                    # raw sensors + flying condition of every flight
u = tf.reshape(u_flat, [-1, T, n_conditions])         # one-hot condition per flight
x = x * (u @ gamma) + (u @ beta)                      # NEW: learned scale & offset PER CONDITION
x = ema_matrix @ x                                    # KePIN's EMA smoothing, now AFTER the condition step
z = self.encode(x)                                    # KePIN's encoder, unchanged
```

Source: simplified from dynamic_koopman/dk_model.py:155-162 (_condition_aware_call)

- **gamma, beta** (one value per condition and sensor) start as plain per-condition normalisation and are then **learned** during training, so this is a trainable layer, not just preprocessing.
- The EMA is a fixed matrix with KePIN's exact smoothing strength, checked to reproduce KePIN's preprocessing to 0.0000004.
- On FD001/FD003 (one flying condition) the layer reduces to a learned normalisation, so no big gain is expected: results were 13.41 / 12.85 on FD001 (KePIN 12.92) and 11.17 / 11.07 on FD003 (KePIN 11.56), none significantly different. It does not hurt where there is nothing to fix.

11. Koopman with Control

In one sentence

The idea was to split the hidden state into "flight effect" + "wear state", but explaining it showed the "wear state" still contains the flying condition; the gain belongs to change 1.

Koopman with control: the idea

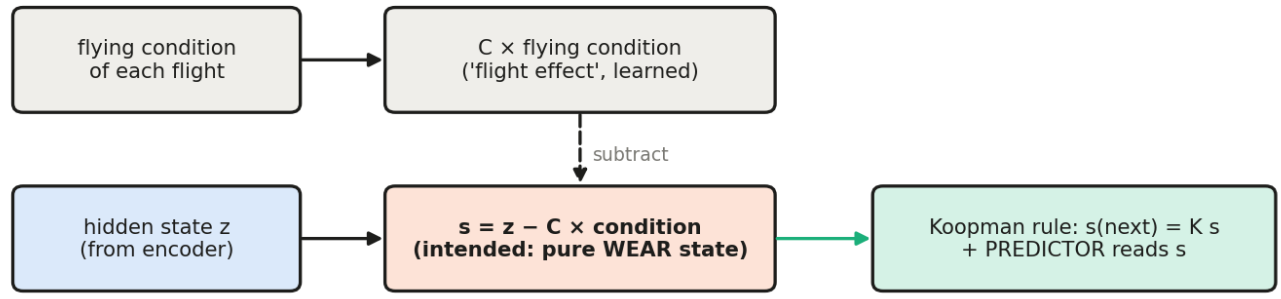


Figure 14. Koopman with control, the standard physics form for a system driven by an external input. The flight effect C × condition is subtracted, and the Koopman rule and predictor work on the remainder s.

How we checked it

We asked simple probes (a linear model and a random forest) how well each hidden state predicts (a) the window's flying conditions and (b) the remaining life, always on unseen engines (FD002, 3 runs).

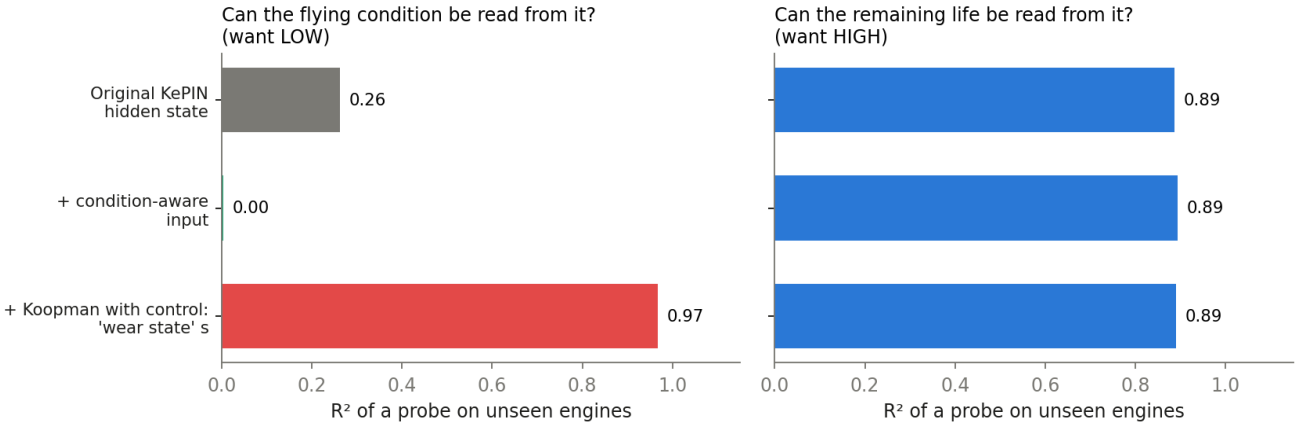


Figure 15. Left: flying-condition information (we want it low). Right: remaining-life information (we want it high).

Why?

The condition-aware input layer works exactly as intended: it removes the flying-condition information from KePIN's hidden state (0.26 → 0.00) and keeps all the remaining-life information (0.89).

Koopman with control does not work as designed. The input layer had already removed the condition, so there was nothing left to subtract. The learned C × condition therefore *wrote the condition back in* as a clean, simple signal (0.97), effectively a per-condition offset.

This is the second time explaining the model caught something RMSE alone would hide (the first was Signal A). For the thesis, the improvement should be credited to the condition-aware input layer.

12. Final Results, Ablation and Limitations

In one sentence

The condition-aware input layer is our main accuracy contribution; the wear model is our main explainability contribution; and several Koopman ideas were tested and honestly ruled out.

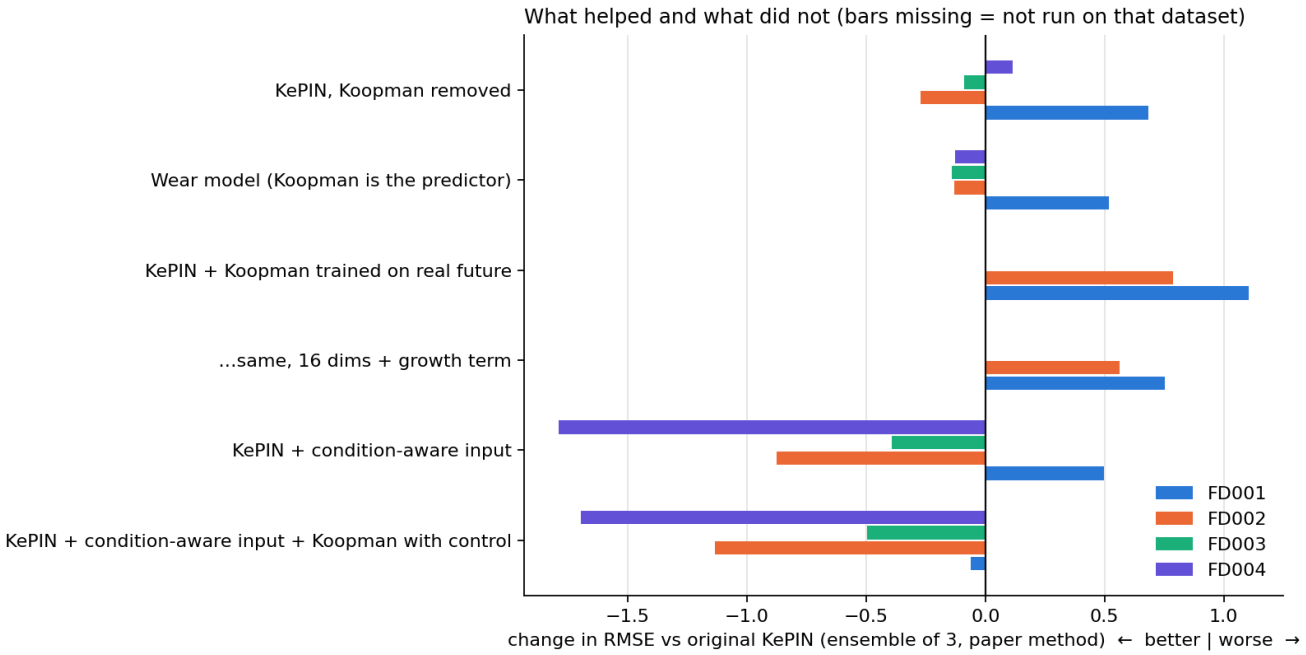


Figure 16. Every experiment compared with the original KePIN (ensemble of 3, paper's method). Left of the line = better. Missing bars = not run on that dataset.

Everything we tried, in one table

Idea	Result vs KePIN	Why
Remove Koopman completely	no significant change (all 4 datasets)	accuracy comes from the encoder
Dynamic Koopman (condition / health / both)	no improvement	re-packaged known information; predictor ignored it
Train Koopman on each engine's real future (5–60 flights ahead)	worse (FD001 14.02, FD002 15.79)	a fading rule cannot describe growing wear, so the shared hidden state was squeezed
Same, on 16 dedicated dims + growth term	still worse (13.67, 15.57)	the extra task competes with the main task
Time-to-failure (Koopman as predictor)	slightly worse; never crosses	a stable rule can only fade
Wear model (TTF + wear accumulator)	ties on all 4 datasets	accurate AND explains every prediction
Condition-aware input layer	**−0.9 to −1.8 (FD002, FD004)**; ties on FD001/FD003	**removes flying-condition noise before smoothing**
Koopman with control (on top)	no reliable extra gain	its "wear state" still carries the condition

What we learned

- **Explainability found the real problem.** Explaining Signal A exposed the flying-condition shortcut; fixing it gave the accuracy gain.
- **KePIN's Koopman part does not drive its accuracy**: constant features, a predictor that ignores it, and a complete removal that changes nothing significantly.
- **Stable Koopman rules cannot represent growing wear**, which is why time-to-failure and future-training failed. A never-fading wear mode fixes it.
- **The paper's evaluation chooses its model with the test set**, which flatters its numbers by about 0.4–0.6.

Limitations (honest)

- Condition-aware KePIN gives big gains only on FD002/FD004 (6 flying conditions); on FD001/FD003 it ties KePIN, because there is no condition noise to remove.
- Koopman with control adds no reliable accuracy and does not produce a condition-free state; it needs a redesign (for example, a penalty that keeps condition information out of s) or should be dropped.
- 3 seeds per model, as in the paper: differences smaller than about 0.5 RMSE cannot be detected reliably.
- Recent results use the paper's method (test set picks the model) for comparability; a fair-test run of the final model would make the claim stronger.
- The accuracy gain comes from how the inputs are handled, not from Koopman itself.

Next steps

- Decide the final model: condition-aware input alone, or with control (slightly better on 3 of 4 datasets, but its "wear state" does not do what it was designed to do).
- Put the wear model on top of condition-aware KePIN: the new accuracy *plus* per-prediction explanations.
- Fair-test (clean) runs of the final model for the thesis.

13. Glossary

Term	Plain meaning
RUL	Remaining Useful Life: flights left before the engine fails.
RMSE	Typical prediction error in flights. Lower is better.
NASA score	An error measure that punishes late predictions (dangerous) more than early ones.
Encoder	The part of the network (convolution + LSTM + attention) that turns sensor data into a hidden state.
Hidden state (z)	The network's internal summary of the engine: a list of 96–128 numbers per flight.
Koopman operator / rule (K)	A learned matrix that moves the hidden state forward one flight: next = K × current.
Mode / eigenvalue	The Koopman rule splits into simple patterns (modes); each fades by its own factor per flight (eigenvalue).
Stable	Every mode fades (factor below 1), so predictions never explode, but nothing can grow.
Wear accumulator	An extra mode with factor exactly 1 that adds the wear rate every flight; it never fades.
Flying (operating) condition	A combination of altitude, speed and throttle; FD002/FD004 have 6.
Normalisation	Rescaling a sensor so values are comparable (subtract an average, divide by a spread).
EMA smoothing	A running average that reduces noise; each value is mostly the previous smoothed value plus a little of the new one.
Seed / run	One training from a random start. We use 3, like the paper.
Ensemble	Average the predictions of the 3 runs, then score once. The paper reports this.
Paper's method vs fair test	Paper: the test set chooses the final model. Fair: held-out training engines choose it.
Ablation	Remove one part and retrain, to measure whether it matters.
p-value	Chance that a difference this large happens by luck. Below 0.05 is usually called significant.
Shapley values / Integrated Gradients	Explainability methods that measure how much each input contributes to an output.
Probe / R^2	A simple model that tests whether some information can be read from a hidden state; $R^2 = 1$ fully, 0 not at all.
Shortcut	When a model learns an easy pattern that happens to work, instead of the intended one.

Code and data for everything in this guide are in `RCKePIN/dynamic_koopman/` *(experiment log: README.md) and* `RCKePIN/FULL_REPORT/REPORT.md`.